Wprowadzenie do Playwright

Porównanie z Cypress

Agenda

1. Wprowadzenie teoretyczne do Playwright
2. Konfiguracja nowego projektu
3. Prezentacja najważniejszych funkcjonalności
4. Porównanie teoretyczne Playwright i Cypress
5. Sytuacja rynkowa i popularność narzędzi
6. Porównanie praktyczne frameworków

Wprowadzenie teoretyczne do Playwright

Co to jest Playwright?

- ⇒ Framework do automatyzacji testów aplikacji webowych
- ⇒ Stworzony przez Microsoft i napisany w TypeScript
- ⇒ Na rynku zadebiutował na początku 2020 roku
- ⇒ Zapewnia możliwość automatyzacji Chromium, Firefox i WebKit
- ⇒ Działa z JavaScript/TypeScript, Python, Java, C#
- ⇒ Wspiera testowanie GUI oraz API

Odmiany Playwright

JS TS
@playwright/test

Najbardziej popularna wersja narzędzia, zawiera wiele wbudowanych funkcji

JS TS
playwright

Biblioteka do obsługi ‘czystego’ Playwright, działająca z różnymi językami programowania

JS TS
playwright-core

Paczka która zawiera rdzeń frameworka, używana zarówno w playwright jak i w @playwright/test

Co oferuje Playwright Test?

- ⇒ Prosta konfiguracja testów
- ⇒ Wbudowane asercje
- ⇒ Wbudowane raporty
- ⇒ UI mode
- ⇒ Trace viewer
- ⇒ Visual testing
- ⇒ Plugin do VS Code

Konfiguracja nowego projektu

npm init playwright@latest

```
~/Downloads/Mentoring  npm init playwright@latest
Getting started with writing end-to-end tests with Playwright:
Initializing project in '.'
✔ Do you want to use TypeScript or JavaScript? · TypeScript
✔ Where to put your end-to-end tests? · tests
✔ Add a GitHub Actions workflow? (y/N) · false
✔ Install Playwright browsers (can be done manually via 'npx playwright install')? (Y/n) · true
Initializing NPM project (npm init -y)…
```

Happy hacking! 🎭

Podstawowa struktura projektu

FOLDERS: TEST

- node_modules
- tests
  - example.spec.ts
- tests-examples
  - demo-todo-app.spec.ts
- .gitignore
- package-lock.json
- package.json
- playwright.config.ts

example.spec.ts

tests > example.spec.ts > ...

```
1  import { test, expect } from '@playwright/test';
2
3  test('has title', async ({ page }) => {
4    await page.goto('https://playwright.dev/');
5
6    // Expect a title "to contain" a substring.
7    await expect(page).toHaveTitle(/Playwright/);
8  });
9
10 test('get started link', async ({ page }) => {
11   await page.goto('https://playwright.dev/');
12
13   // Click the get started link.
14   await page.getByRole('link', { name: 'Get started' }).click();
15
16   // Expects page to have a heading with the name of Installation.
17   await expect(page.getByRole('heading', { name: 'Installation' })).toBeVisible();
18 });
```

Prezentacja najważniejszych funkcjonalności

- Prosta konfiguracja testów
- Wbudowane asercje
- Wbudowane raporty
- UI mode
- Trace viewer
- Visual testing
- Plugin do VS Code

Porównanie teoretyczne Playwright i Cypress

CREATIVE PARTNER IN TECHNOLOGY

Playwright

Zalety	Wady
⇒ Składnia async/await	⇒ Składnia async/await
⇒ Łatwa obsługa okien, tabów, iframe	⇒ Brak wsparcia dla starszych wersji przeglądarek
⇒ Wsparcie dla różnych języków programowania	⇒ Brak natywnego wsparcia dla BDD - Cucumber framework
⇒ Playwright Test	⇒ Brak wsparcia dla urządzeń mobilnych (tylko symulatory) - BrowserStack

Cypress

Zalety	Wady
⇒ Stosunkowo prosta i czytelna składnia	⇒ Utrudnione testowanie tabów, okien, obcych domen
⇒ UI mode zapewniający przyjemne debugowanie testów	⇒ Działa tylko z JavaScript/TypeScript
⇒ Dobra dokumentacja i duże community	⇒ Niektóre funkcjonalności są płatne np. paralelizacja
⇒ Płatna usługa Cypress Cloud zapewniająca wiele zaawansowanych funkcji	⇒ Od V13 wprowadzone zostało ograniczenie w używaniu pluginów

Zestawienie

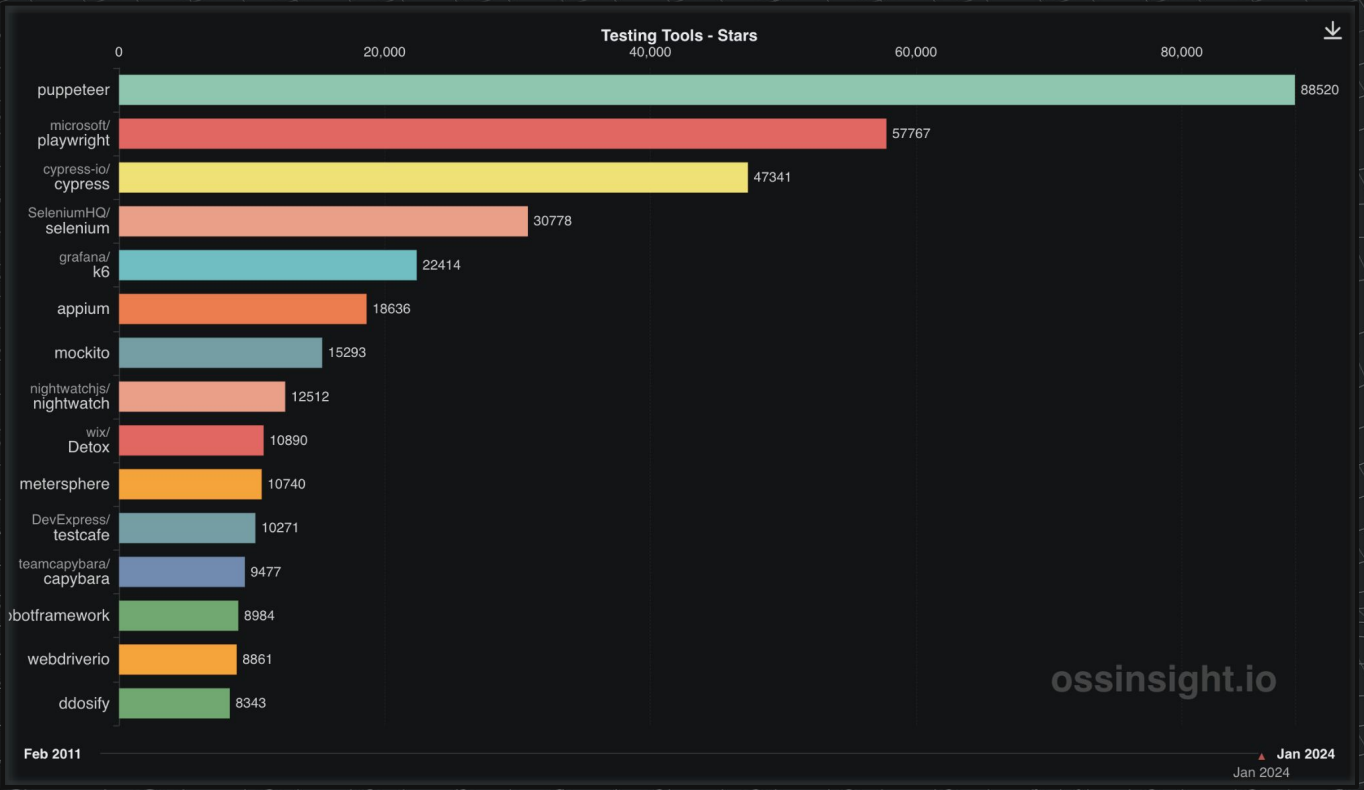
Kryteria	Cypress	Playwright
Języki programowania	JavaScript	JavaScript, Python, Java, C#
Test runner	Mocha	Wbudowany, Mocha, Jest, Jasmine
Przeglądarki	Chrome, Firefox, Edge, Safari (experimental)	Chromium, Firefox, WebKit
Architektura	Uruchamia testy bezpośrednio w przeglądarce	Event-driven architecture - reaguje na zdarzenia w przeglądarce
Wsparcie urządzeń mobilnych	Wspiera testowanie na fizycznych urządzeniach i rozwiązania chmurowe	Wspiera tylko symulatory
Obecność na rynku	7 lat	4 lata
Online dashboard	Tak	Nie
CI / CD	Wszystkie popularne platformy, np. Github Actions, Bitbucket, AWS	Wszystkie popularne platformy, np. Jenkins, Gitlab, Azure DevOps

Sytuacja rynkowa i popularność narzędzi

Chronologia frameworków

2004	Selenium
2012	Appium
2015	Detox
2017	Cypress
2018	Puppeteer
2020	Playwright

Popularność poszczególnych narzędzi



Źródło: ossinsight.io

Liczba pobrań w ostatnich 2 latach

Źródło: npmtrends.com

Sytuacja rynkowa Cypress

- 2020 - Cypress dostał potężne wsparcie od inwestorów
- 2023 - Zespół Cypress dotknęły zwolnienia
- 2023 - Od V13 wprowadzone zostały ograniczenia w wykorzystywaniu zewnętrznych usług np. Sorry Cypress i Currents

---

- Podobno płatne usługi cieszą się małym zainteresowaniem - brak oficjalnej informacji
- Nie zapowiada się aby Cypress w najbliższym czasie zniknął z rynku

Porównanie praktyczne frameworków

CREATIVE PARTNER IN TECHNOLOGY

Linkografia

1. https://playwright.dev/docs/intro
2. https://www.lambdatest.com/blog/cypress-vs-playwright
3. https://playwright.info/cypress-dying-so-use-playwright
4. https://playwright.info/cypress-playwright-statystyki-popularnosc
5. https://jaktestowac.pl/bughuntfest
6. https://www.youtube.com/watch?v=SRvRdFZ1DO8&ab_channel=FabrykaTest%C3%B3w&t=62m30s